Delphi Web Studio Runtime Flow

How the Windows application starts, processes a project, and finishes a session

Last changed: November 1, 2026

Based on the current project source code. Intended for users, testers, and maintainers.

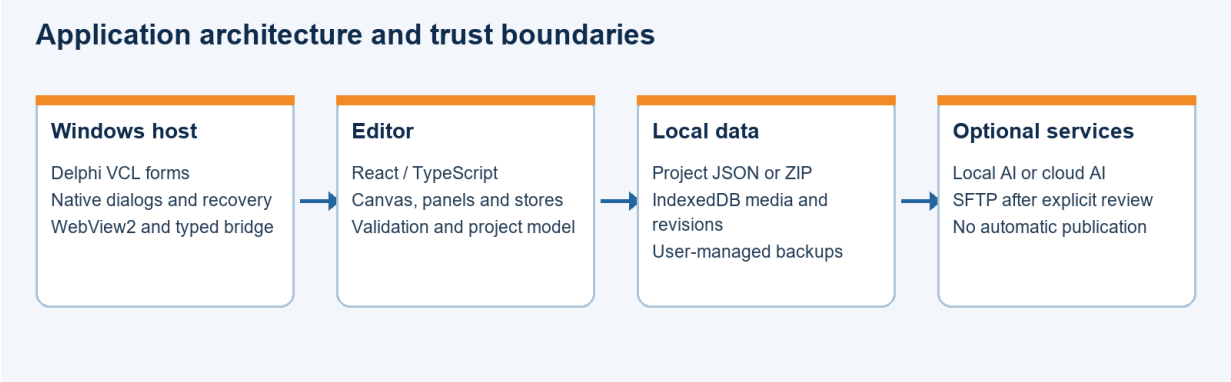


Figure 1. The Windows host, local editor, and optional external services.

Contents

Purpose and scope 3
Runtime overview 3
1 Start the Windows host 3
2 Initialize the local editor 4
3 Select or open project data 4
4 Enter the editor and change the project 4
5 Authorize native operations 5
6 Save, revisions, and recovery 5
7 Review and export the website 6
8 Optional AI and external services 6
9 Close the application 7
Failure and decision points 7
Source map and verification limits 7

Purpose and scope

This document follows the program while it runs, from the Windows process starting through the end of a session. It describes the normal path, important decisions, the data produced, and the failure paths visible in the current implementation. The User Guide explains how to operate each control; the Engineering Guide contains the wider implementation reference.

A project is editable application data. A website export is a separate set of output files. Saving a project, reviewing an export, and publishing a website are distinct actions. Opening the application does not publish a site.

Runtime overview

Windows starts the Delphi VCL executable. The main form creates a WebView2 browser for the local editor. React then opens the dashboard and manages project and editor state. User actions select the branches for opening, editing, importing, saving, exporting, or using optional services. The application remains active until the window is closed.

The diagram is a conceptual sequence. The paths are conditional: a user may save several times, return to editing after an export, or close without using network services.



Figure 2. The project lifecycle at a glance.

1 Start the Windows host

DelphiWebStudio.dpr initializes the VCL application, creates TMainForm, and enters the application message loop. The main form checks for editor\index.html next to the executable. If the file is missing, it reports that the local editor files must be built and staged; it does not silently fetch a website.

If the files exist, the form requests WebView2 creation. A WebView2 initialization or local mapping failure is displayed in the status area. The machine needs a working compatible WebView2 runtime and the editor files shipped with the executable.

Source: desktop/DelphiWebStudio/DelphiWebStudio.dpr; desktop/DelphiWebStudio/WebStudio.Main.pas, FormCreate and EditorBrowserCreateWebViewCompleted.

2 Initialize the local editor

After WebView2 is created, the host enables web messages and maps the bundled editor folder to https://webstudio.local. It navigates to the local index page. This virtual host name identifies files served from the local folder; its use does not mean that the project is publicly hosted.

The host restricts navigation to the local origin, cancels frame navigation, handles requests for new windows, and controls browser permissions. The editor adds a content security policy for the local origin, mounts the React application, and uses hash-based routing in the desktop host. The dashboard is the initial route.

Source: WebStudio.Main.pas, WebView event handlers; src/main.tsx; src/App.tsx.

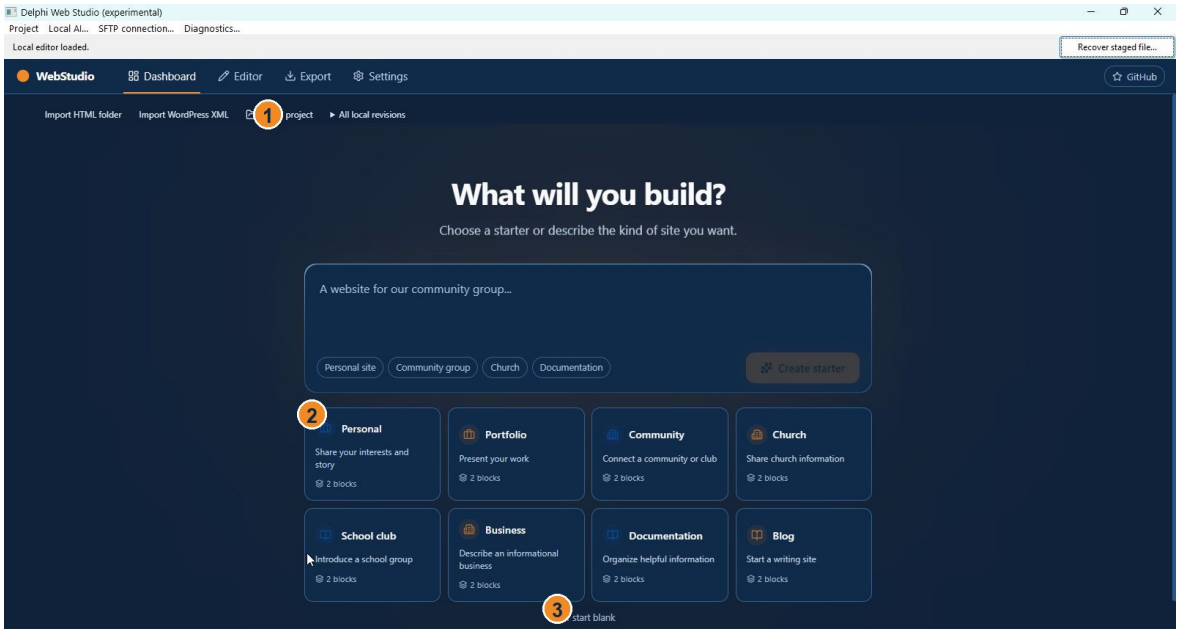


Figure 3. The dashboard is the entry point to project actions.

3 Select or open project data

From the dashboard, a user can start with a blank project or starter, or select an existing supported project. The open flow checks the selected file and decodes its project data. Supported legacy data is migrated in memory; a source file is not silently rewritten merely because it was opened. Validation errors stop the open operation and leave the original available.

HTML folder and WordPress XML import follow separate preview and conversion paths. They create a separate project after the user accepts the conversion. Import does not recreate every original layout, plugin, script, or server capability. The result must be reviewed.

Source: src/routes/Dashboard.tsx; src/project/ProjectFileCommands.tsx; src/project/project-file.ts; src/project/static-html-import.ts; src/project/wordpress-import.ts.

4 Enter the editor and change the project

The editor renders the active page from project configuration and displays the selected block, its properties, and page controls. Editing actions update application state; page and block operations change that configuration. Media operations use the project media library. Preview displays the current project; it is not proof that the published website will behave identically in every browser.

The browser profile can retain local drafts and project data. The implementation protects unreadable or incompatible stored drafts from being overwritten in specific recovery cases. Local draft storage is not a substitute for an independently saved project file.

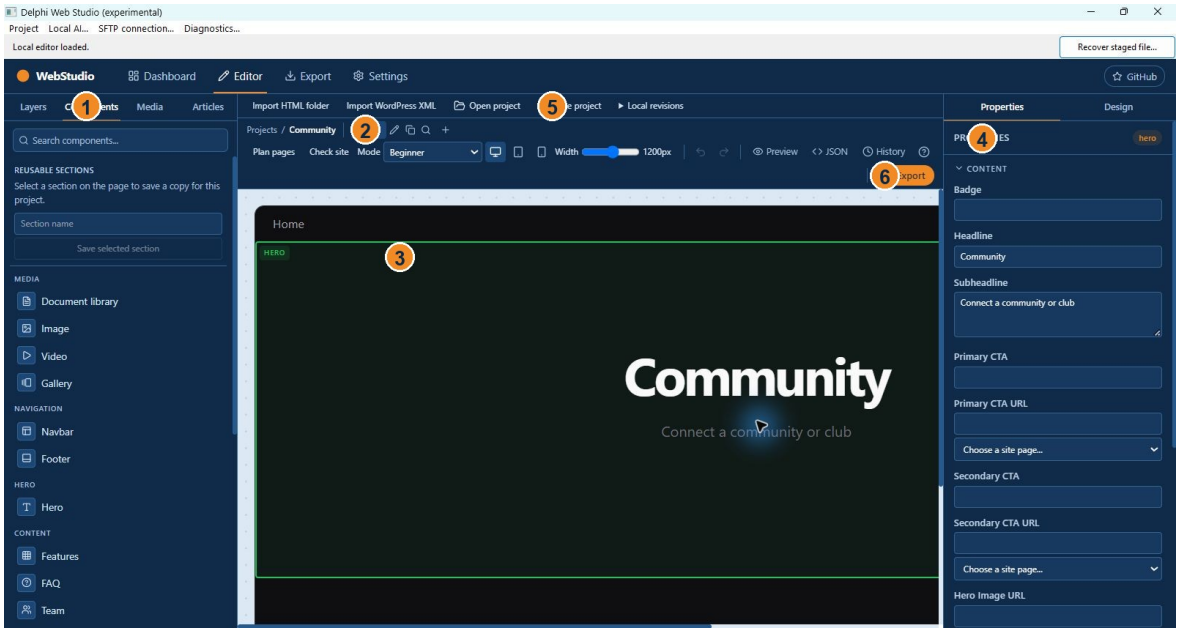


Figure 4. The editor combines page selection, canvas, and properties.

Source: src/routes/Editor.tsx; src/store/configStore.ts; src/store/projectsStore.ts; src/media/media-library.ts.

5 Authorize native operations

When the desktop editor initiates an operation that requires a native capability, it requests a short-lived session for the specific project and action. The JavaScript bridge exchanges a session request and a capability request with the host. The host checks message origin and format, project identity, capability, and session validity. The bridge does not transfer project file paths or file content in those authorization messages.

The native host handles project.open, project.save, media.import, and site.export capabilities. Invalid or untrusted requests are rejected. Navigating away revokes the current bridge session. A failed or timed-out request stops the corresponding action rather than granting access.

Source: src/desktop/native-bridge.ts; desktop/DelphiWebStudio/WebStudio.Main.pas, EditorBrowserWebMessageReceived; desktop/bridge.

6 Save, revisions, and recovery

A save creates a snapshot of the project. The editor validates and serializes the project; media-bearing projects can be packaged as ZIP files. A user chooses the destination through the save flow. The desktop host handles the download, staging, and completion. A save in progress can delay window closure; closing again can interrupt it.

For replacement, the native project-save path includes a recovery backup and checks before committing the staged file. Failure can leave a recovery or staged file for inspection. If the project changes while a snapshot is being prepared, the saved snapshot may not contain later changes; follow any prompt to save again.

Local revisions and separate backup ZIPs are additional recovery options. A local revision belongs to local application storage and is not a substitute for an independent backup. Restoring a backup creates a selected destination file; open the restored file to verify it.

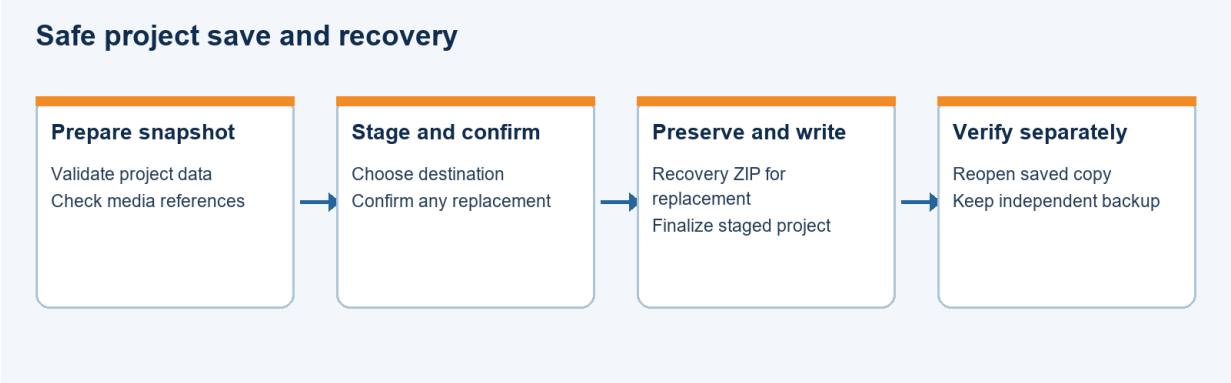


Figure 5. Project saving and independent verification are separate operations.

Source: src/project/ProjectFileCommands.tsx, saveProject; desktop/DelphiWebStudio/WebStudio.Main.pas, EditorBrowserDownloadStarting and backup/restore handlers; desktop/project.

7 Review and export the website

The export flow uses project configuration and available media to prepare a standalone HTML file or a site ZIP, depending on the selected option. The review flow can show the files and manifest before a download. If the project changes after review, the user must review the new version again before downloading that reviewed output.

Export produces local files. It does not by itself set up GitHub Pages or publish a website. Check the exported pages, links, media, forms, and layout before any separately authorized hosting or transfer.

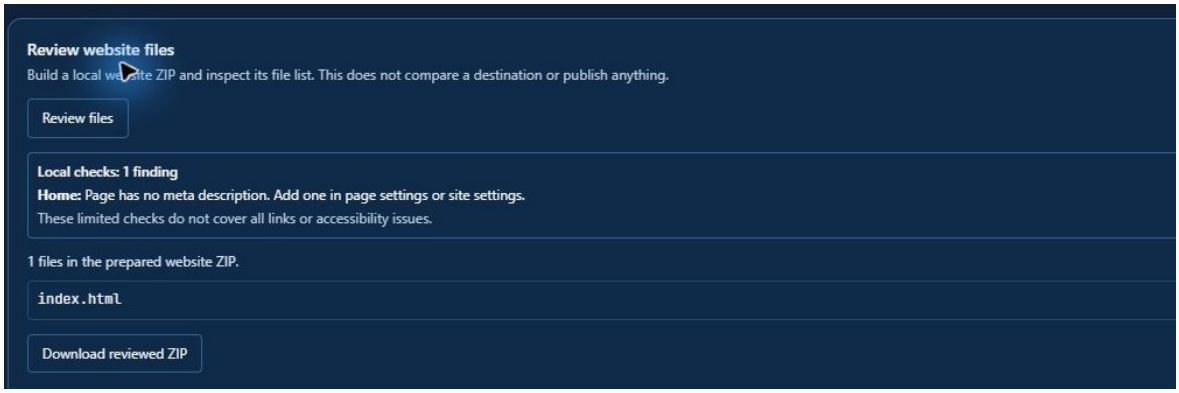


Figure 6. Reviewing output files is separate from publishing.

Source: src/routes/Deploy.tsx; src/lib/export-html.ts.

8 Optional AI and external services

Local AI is a user-selected path. The native dialog collects runtime archive, model, brief, and proposal type. Processing runs in a background thread; completion or failure is returned to the editor. The user reviews proposed changes. The main window prevents closure while this work is active.

Cloud AI is an optional network path. Its use depends on the selected configuration and user consent; prompts may be sent to the configured service. A new session key is handled as described in the User Guide. Generated content requires review.

SFTP and other deployment features are separate from ordinary local editing and export. Connection, review, transfer, and hosting are not implied by starting or saving a project. An external transfer or publication needs the appropriate deliberate user action.

Source: desktop/DelphiWebStudio/WebStudio.Main.pas, LocalAIItemClick; src/lib/generate-site.ts; desktop/DelphiWebStudio/WebStudio.SftpForm.pas.

9 Close the application

On a normal close, the VCL form checks for local AI work and active downloads or project saves. It blocks closure while local AI is running. For a save or download, the form warns and permits a subsequent close to interrupt it. When the form is destroyed, the native bridge session is revoked and released.

A completed save should still be reopened or otherwise verified when the contents are important. Closing the window does not independently guarantee that the latest edits were saved to a portable project file.

Source: desktop/DelphiWebStudio/WebStudio.Main.pas, FormCloseQuery and FormDestroy.

Failure and decision points

Startup: missing editor files, an unavailable WebView2 runtime, or failed local mapping prevent the editor from loading. Read the status message and retain existing projects.

Open or import: an unsupported, invalid, damaged, or too-large input can be rejected. Keep the original and inspect the reported error. Import previews can show incomplete conversion.

Save or export: a denied capability, cancelled destination, invalid data, missing media, interrupted download, or failed file operation can stop output. Do not assume the target contains a complete current project; retain any indicated staged or recovery file.

Optional operations: local AI can fail to initialize its selected runtime or model; network services can fail or require configuration. These branches are not required for ordinary local editing.

Source map and verification limits

The runtime sequence was assembled from the project entry point, the native WebView2 host, the React entry and routes, the project commands, state stores, export code, and the optional-service handlers named in the sections above. The diagrams and screenshots illustrate the flow; they are not a trace of a particular user session.

The document describes the source available at the time of revision. Clean Windows installation and publication-dependent acceptance have been deferred; this document does not claim those tests have passed. UI labels and behavior can change with future releases.